SQL TASK 2

Data Aggregation & Reporting

Complete Solution - Queries, Explanations & Sample Output

About this document

This solution works through four aggregation and reporting exercises on the **Employees Sample Database** (MySQL). The source describes the **salaries**, **employees**, **departments**, and **dept_emp** tables.

> **Note:** Output tables reproduce sample values from the uploaded task PDF. They are not independently verified against a live database.

What's inside

01 **Department Salary Summary**
Counts, average, min, max and range per department

02 **Yearly Hiring Trends**
Hires per year with YoY change and growth rate

03 **High-Earning Departments**
Filtering aggregated groups with HAVING

04 **Salary Distribution Report**
10,000 salary buckets and percentage share

QA **Validation, Quality Checks & Common Errors**
Sanity checks and typical pitfalls

EXERCISE 1

Department Salary Summary

Aim: Report employee counts, average salary, minimum salary, maximum salary, and salary range for each department.

```
SELECT
  d.dept_name                              AS Department,
  COUNT(DISTINCT e.emp_no)                 AS Employees,
  ROUND(AVG(s.salary), 2)                  AS Avg_Salary,
  FORMAT(MIN(s.salary), 0)                 AS Min_Salary,
  FORMAT(MAX(s.salary), 0)                 AS Max_Salary,
  FORMAT(MAX(s.salary) - MIN(s.salary), 0) AS Salary_Range
FROM departments d
JOIN dept_emp  de ON d.dept_no = de.dept_no
JOIN employees e  ON de.emp_no = e.emp_no
JOIN salaries  s  ON e.emp_no  = s.emp_no
WHERE de.to_date > CURDATE()
  AND s.to_date  > CURDATE()
GROUP BY d.dept_name
ORDER BY Avg_Salary DESC;
```

How it works

JOIN connects the tables; **COUNT(DISTINCT)** counts unique employees; **AVG**, **MIN**, and **MAX** summarize salaries; **Salary_Range** is maximum minus minimum; **GROUP BY** creates one row per department; **ORDER BY** sorts by average salary, highest first.

Sample output

Department	Employees	Avg_Salary	Min_Salary	Max_Salary	Salary_Range
Sales	37,240	88,770.63	40,238	158,220	117,982
Marketing	14,844	71,952.13	39,834	141,953	102,119
Finance	12,377	70,443.87	40,850	142,395	101,545
Research	15,442	67,865.10	38,823	141,536	102,713
Production	51,328	67,479.79	38,954	138,273	99,319

EXERCISE 2

Yearly Hiring Trends

Aim: Count hires by year and calculate the year-over-year change and growth rate.

```
WITH yearly_hires AS (
  SELECT YEAR(hire_date) AS Hire_Year,
         COUNT(*)        AS New_Hires
  FROM employees
  GROUP BY YEAR(hire_date)
),
hiring_changes AS (
  SELECT Hire_Year,
         New_Hires,
         New_Hires - LAG(New_Hires) OVER (
           ORDER BY Hire_Year
         ) AS YoY_Change,
         LAG(New_Hires) OVER (
           ORDER BY Hire_Year
         ) AS Previous_Hires
  FROM yearly_hires
)
SELECT
  Hire_Year,
  New_Hires,
  YoY_Change,
  CONCAT(
    ROUND(100.0 * YoY_Change /
          NULLIF(Previous_Hires, 0), 1),
    '%'
  ) AS Growth_Rate
FROM hiring_changes
WHERE Hire_Year BETWEEN 1985 AND 1995
ORDER BY Hire_Year;
```

How it works

YEAR() extracts the year; **COUNT(*)** counts hires; **LAG()** gets the previous year's count. **YoY_Change** = current hires - previous hires. **Growth_Rate** = change / previous hires x 100. **NULLIF** avoids division by zero.

Sample output

Hire_Year	New_Hires	YoY_Change	Growth_Rate
1985	35,361	NULL	NULL
1986	36,149	788	2.2%
1987	34,876	-1,273	-3.5%
1988	35,083	207	0.6%

The source PDF shows these sample years and values; the full query can return additional years through 1995.

EXERCISE 3

High-Earning Departments (HAVING)

Aim: Show departments with average salary greater than 70,000 and count greater than 10,000.

```
SELECT
  d.dept_name,
  ROUND(AVG(s.salary)) AS avg_salary,
  COUNT(*)             AS employees
FROM departments d
JOIN dept_emp de ON d.dept_no = de.dept_no
JOIN salaries s  ON de.emp_no = s.emp_no
WHERE s.to_date > CURDATE()
GROUP BY d.dept_name
HAVING AVG(s.salary) > 70000
   AND COUNT(*) > 10000;
```

How it works

WHERE filters rows before grouping. **HAVING** filters aggregated groups after GROUP BY. Both HAVING conditions must be true.

Sample output

dept_name	avg_salary	employees
Sales	88,771	37,240
Marketing	71,952	14,844
Finance	70,444	12,377

EXERCISE 4

Salary Distribution Report

Aim: Group current Sales salaries into 10,000 salary buckets and calculate each bucket's percentage of the department's total.

```
SELECT
  d.dept_name,
  FLOOR(s.salary / 10000) * 10000 AS Salary_Bucket,
  COUNT(*)                        AS Employees,
  CONCAT(
    ROUND(
      100.0 * COUNT(*) /
      SUM(COUNT(*)) OVER (
        PARTITION BY d.dept_name
      ),
      1
    ),
    '%'
  ) AS Distribution
FROM departments d
JOIN dept_emp de ON d.dept_no = de.dept_no
JOIN salaries s  ON de.emp_no = s.emp_no
WHERE s.to_date > CURDATE()
  AND d.dept_name = 'Sales'
GROUP BY d.dept_name, Salary_Bucket
ORDER BY Salary_Bucket;
```

How it works

FLOOR(salary / 10000) * 10000 creates the bucket's lower boundary. **COUNT(*)** counts rows in each bucket. The window **SUM** of grouped counts gives the total for the department. Distribution is bucket count divided by total x 100.

Sample output

Department	Salary_Bucket	Employees	Distribution
Sales	40000	3,521	9.5%
Sales	50000	9,842	26.4%
Sales	60000	10,115	27.2%
Sales	70000	6,884	18.5%

QUALITY ASSURANCE

Validation & Quality Checks

1. Verify current employees represented in salary records

```
SELECT COUNT(DISTINCT emp_no)
FROM salaries
WHERE to_date > CURDATE();
```

The source says this should be compared with the employee totals in reports.

2. Check for missing department numbers

```
SELECT COUNT(*)
FROM dept_emp
WHERE dept_no IS NULL;
```

If the result is greater than zero, investigate missing department assignments.

PITFALLS

Common Errors & Solutions

1. WHERE vs HAVING

WHERE filters individual records; **HAVING** filters aggregated groups. To filter groups by average salary:

```
SELECT emp_no, AVG(salary) AS avg_salary
FROM salaries
GROUP BY emp_no
HAVING AVG(salary) > 50000;
```

2. Grouping columns

Every selected column that is not aggregated should be included in **GROUP BY** (unless the database can prove functional dependency).

```
SELECT d.dept_name, e.gender, AVG(s.salary) AS avg_salary
FROM departments d
JOIN dept_emp  de ON d.dept_no = de.dept_no
JOIN employees e  ON de.emp_no = e.emp_no
JOIN salaries  s  ON s.emp_no  = e.emp_no
GROUP BY d.dept_name, e.gender;
```

3. Avoid division by zero

```
SELECT SUM(revenue) / NULLIF(COUNT(*), 0)
FROM sales;
```

NULLIF(COUNT(*), 0) returns NULL when the count is zero, avoiding a division-by-zero error.

CONCLUSION

This task practises aggregation and reporting with **COUNT()**, **AVG()**, **MIN()**, **MAX()**, **GROUP BY**, **HAVING**, **JOINs**, **LAG()**, and window functions. Validate totals against the source dataset before using reports for decisions.

Source note: The sample output values are transcribed from the uploaded Task 2 PDF and are not independently verified. Query results can vary depending on dataset version, date, and table contents.